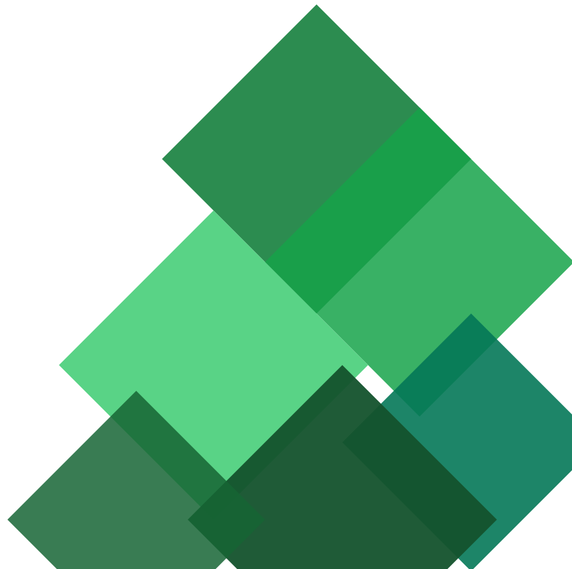
INSTITUTO FEDERAL
Rio Grande do Sul

Comunica Sertão

Desenvolvimento da Infraestrutura Back-End e Integração Municipal

Autor: Fabricio Cardoso de Abreu

Orientadora: Caroline Guterres Silva



Problema e Justificativa

Cenário e Problema

- **Área Rural Extensa:** Dificuldade logística de deslocamento presencial da população até a sede da prefeitura;
- **Ausência de Centralização Transacional:** Falta de uma API unificada para receber e persistir chamados municipais;
- **Vulnerabilidade de Dados:** Demandas registradas de modo informal, sem geração de protocolos formais e sem auditoria pública.

Problema e Justificativa

Justificativa da Solução

- **Processamento Assíncrono Nativo:** API RESTful leve em Python com FastAPI, garantindo tempos de resposta imediatos;
- **Persistência Relacional Estruturada:** Banco de dados relacional SQLite local com ACID gerenciado via SQLAlchemy ORM;
- **Soberania e Custo Zero:** Eliminação de mensalidades proprietárias e retenção segura dos dados dos munícipes na prefeitura.

Objetivos do Projeto

API RESTful

Endpoints estruturados para integração web e mensageria

Webhook WhatsApp

Recepção de mensagens e máquina de estados do bot

Persistência SQLite

Armazenamento relacional de ocorrências e munícipes

Gestão de Acessos

Autenticação institucional com aprovação de novos servidores

Geração de Protocolos

Emissão de códigos únicos (CMS-2026-XXXX) e relatórios

Trabalhos Relacionados

Sistema	Canal de Entrada	Arquitetura Back-End / Dados	Pontos Positivos	Limitações
OuvERJ	Portal Web formal	Monolítico estadual legado	Controle formal de SLA	Alto consumo de infra; sem API para mensageria
156Web	Portal, App e Bot	Central multicanal proprietária	Suporte a filas e roteamento	Exige download; APIs e banco fechados de alto custo
Eaí Prefeito	Chatbot WhatsApp	Servidor comercial em nuvem	Painel gerencial pronto	Cobrança mensal por operador; dados em servidores de terceiros
Comunica Sertão (Back-End)	**WhatsApp Direto / Webhook**	**FastAPI + SQLite + SQLAlchemy**	**Baixa latência, código aberto, soberania e custo zero**	**Depende da conectividade e disponibilidade do host**

Backlog e Prioridades (RFs e RNFs)

Prioridade Alta (Essenciais): RF01 (Recepção de Webhook WhatsApp), RF02 (Máquina de estados e triagem do bot), RF03 (Geração e cálculo de protocolo anual), RF05 (Endpoints REST de ocorrências), RF06 (Autenticação e aprovação de servidores).

Prioridade Média (Gestão): RF04 (Consulta de andamento por protocolo), RF07 (Upload e armazenamento local de fotos), RF08 (Filtros por status, categoria e bairro), RF09 (Consolidação de dados para relatórios gerenciais).

Prioridade Baixa (Complementares): RF10 (Notificações ativas via WhatsApp), RF11 (Auditoria e log de alterações de status), RF12 (Métricas agregadas de tempo médio de atendimento).

Backlog e Prioridades (RFs e RNFs)

RNF01 (Alta): Desempenho e Baixa Latência — Tempo de resposta da API inferior a 200ms para requisições concorrentes e processamento assíncrono de mensagens.

RNF02 (Alta): Persistência Relacional Confiável — Banco de dados SQLite estruturado sob propriedades ACID via SQLAlchemy, garantindo integridade referencial sem demandar SGBDs pesados.

RNF03 (Média): Validação Forte e Documentação Automática — Sanitização de entradas com esquemas Pydantic e geração nativa de especificações OpenAPI/Swagger UI.

Diagrama de Caso de Uso

Cidadão (WhatsApp / Webhook)

01: Enviar mensagem para atendimento do bot;

«extend»: Anexar evidência fotográfica;

02: Consultar andamento por protocolo;

03: Receber confirmação com protocolo oficial.

Servidor (Painel Web / API REST)

04: Autenticar com e-mail institucional;

«extend»: Solicitar acesso / aguardar aprovação;

05: Consultar e filtrar chamados por status;

06: Atualizar status do chamado (Kanban/Tabela);

07: Aprovar ou bloquear servidores (Admin);

08: Obter dados consolidados para relatórios.

Diagrama de Caso de Uso

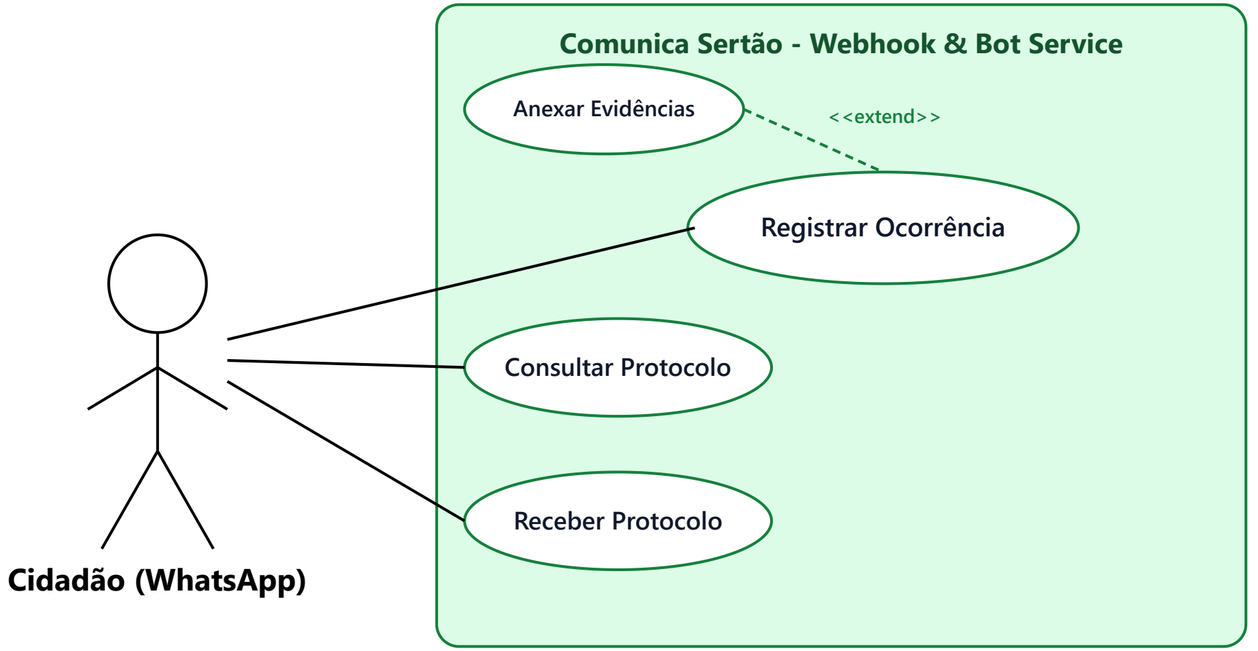


Diagrama de Caso de Uso

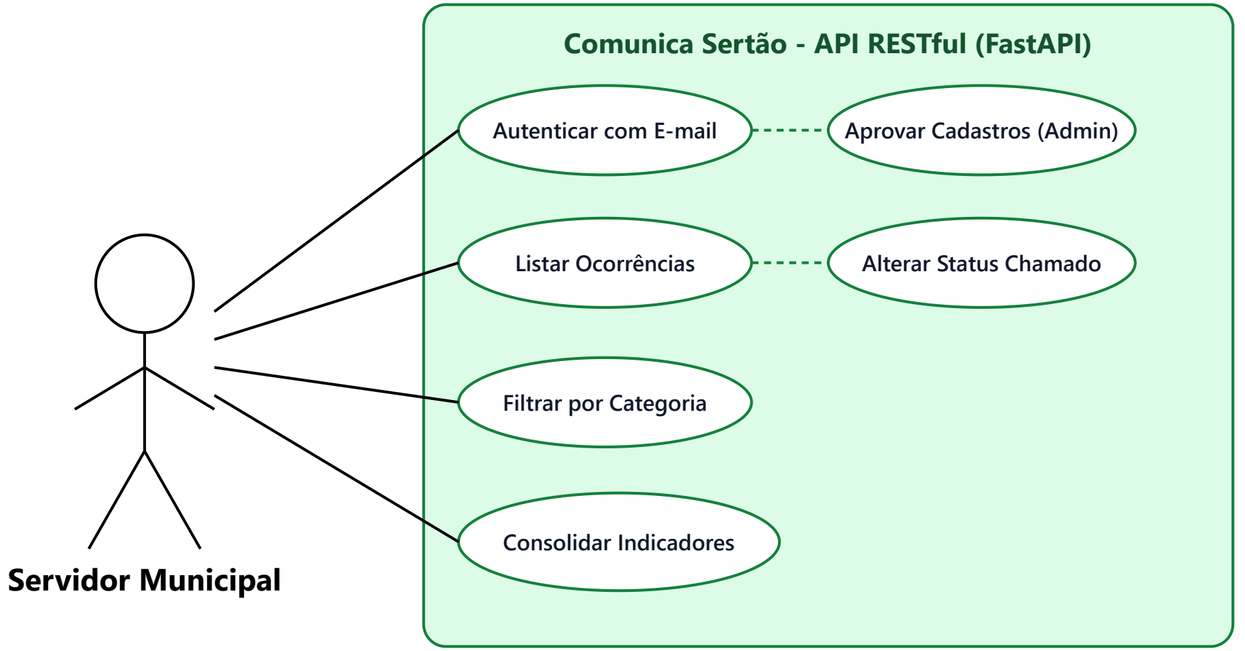


Diagrama Entidade-Relacionamento

Cidadão

id (PK)
nome
telefone_whatsapp
data_cadastro

Secretaria

id (PK)
nome_secretaria
nome_responsavel

Categoria

id (PK)
nome_categoria
secretaria_id (FK)

Servidor

id (PK)
email, cargo
senha_hash
status (Aprovado)

Anexo

id (PK)
ocorrencia_id (FK)
url_arquivo
tipo_arquivo

Histórico_Status

id (PK)
ocorrencia_id (FK)
servidor_id (FK)
status_novo, data

Diagrama Entidade-Relacionamento

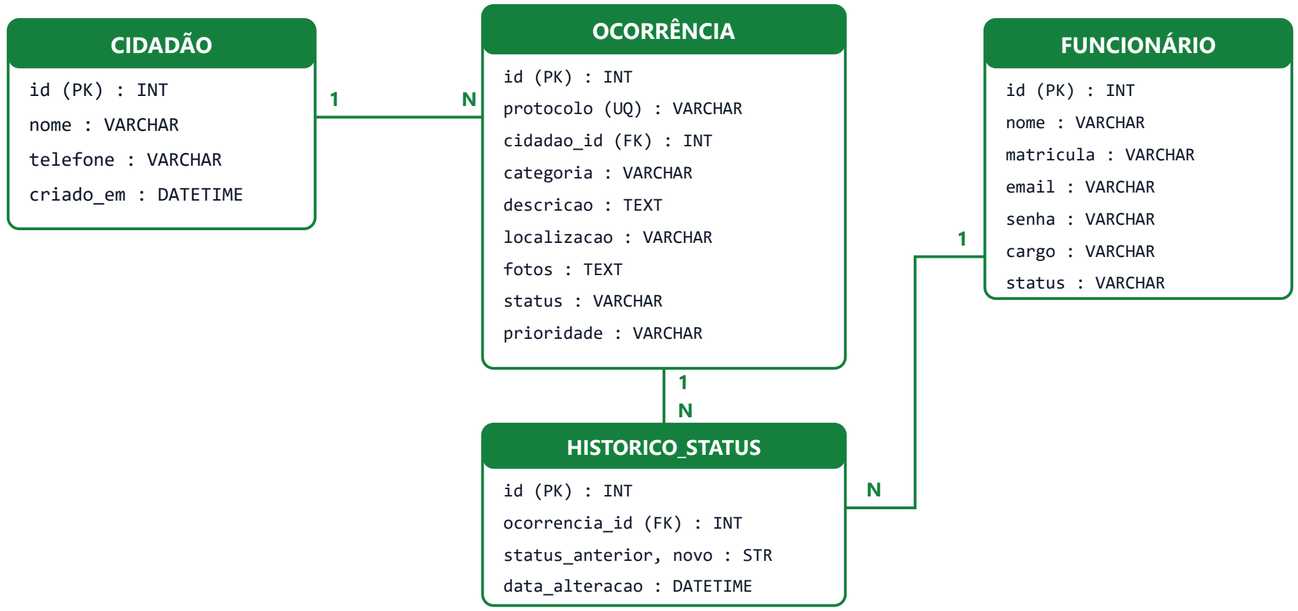
Funcionário (Acessos)

- **id:** Integer (PK, Autoincrement)
- **nome:** String (Nome do Servidor)
- **matricula:** String (Matrícula Municipal)
- **email:** String (@sertao.rs.gov.br)
- **senha:** String (Hash de Segurança)
- **cargo:** String (Administrador / Servidor)
- **status:** String (Aprovado / Pendente / Bloqueado)

Ocorrência (Transacional)

- **id:** Integer (PK, Autoincrement)
- **protocolo:** String (CMS-2026-XXXX, Unique)
- **cidadao_nome:** String
- **cidadao_telefone:** String (WhatsApp)
- **categoria:** String (Iluminação, Obras, etc.)
- **descricao:** Text (Relato do Cidadão)
- **localizacao:** String (Bairro / Rua)
- **fotos:** Text (URLs de Mídia Local)
- **status:** String (Pendente / Em Andamento / Concluído)
- **prioridade:** String (BAIXA / MEDIA / ALTA)
- **criado_em:** DateTime (Timestamp Automático)

Diagrama Entidade-Relacionamento



Tecnologias Utilizadas

(Arquitetura Back-End Web)

FastAPI

Framework web moderno e de alto rendimento construído em Python, baseado em padrões abertos e tipagem estática.

Proporciona tempo de resposta na ordem de milissegundos e geração nativa da documentação interativa OpenAPI/Swagger UI, atendendo ao RNF01 e RNF03.

Tecnologias Utilizadas

(Arquitetura Back-End Web)

Python 3 + Uvicorn

Linguagem de alto nível combinada com servidor ASGI ultrarrápido para processamento concorrente de requisições web.

Garante estabilidade no despacho de webhooks assíncronos e mínimo consumo de memória em servidores municipais.

Tecnologias Utilizadas

(Arquitetura Back-End Web)

SQLAlchemy + SQLite

Mapeamento Objeto-Relacional (ORM) integrado a banco relacional embutido e estruturado (comunicasertao.db).

Assegura integridade transacional (ACID), facilidade de manutenção e cópias de segurança sem dependência de serviços externos pesados (RNF02).

Tecnologias Utilizadas

(Validação e Comunicação)

Pydantic Schemas

Validação e higienização rigorosa dos dados de entrada e saída das rotas da API em tempo de execução.

Impede o recebimento de dados corrompidos ou maliciosos e automatiza a conversão e formatação de respostas JSON.

Tecnologias Utilizadas

(Documentação e Integração)

Swagger UI & Webhooks

Documentação interativa acessível via rota /docs para teste em tempo real de requisições e respostas.

Roteamento assíncrono para recepção dos webhooks do WhatsApp e compatibilidade com a Meta Cloud API.

Prototipação e Implementação

Documentação OpenAPI e Autenticação



Documentação interativa OpenAPI e rotas de controle de acesso institucional.

Prototipação e Implementação

Endpoints de Ocorrências e Métricas

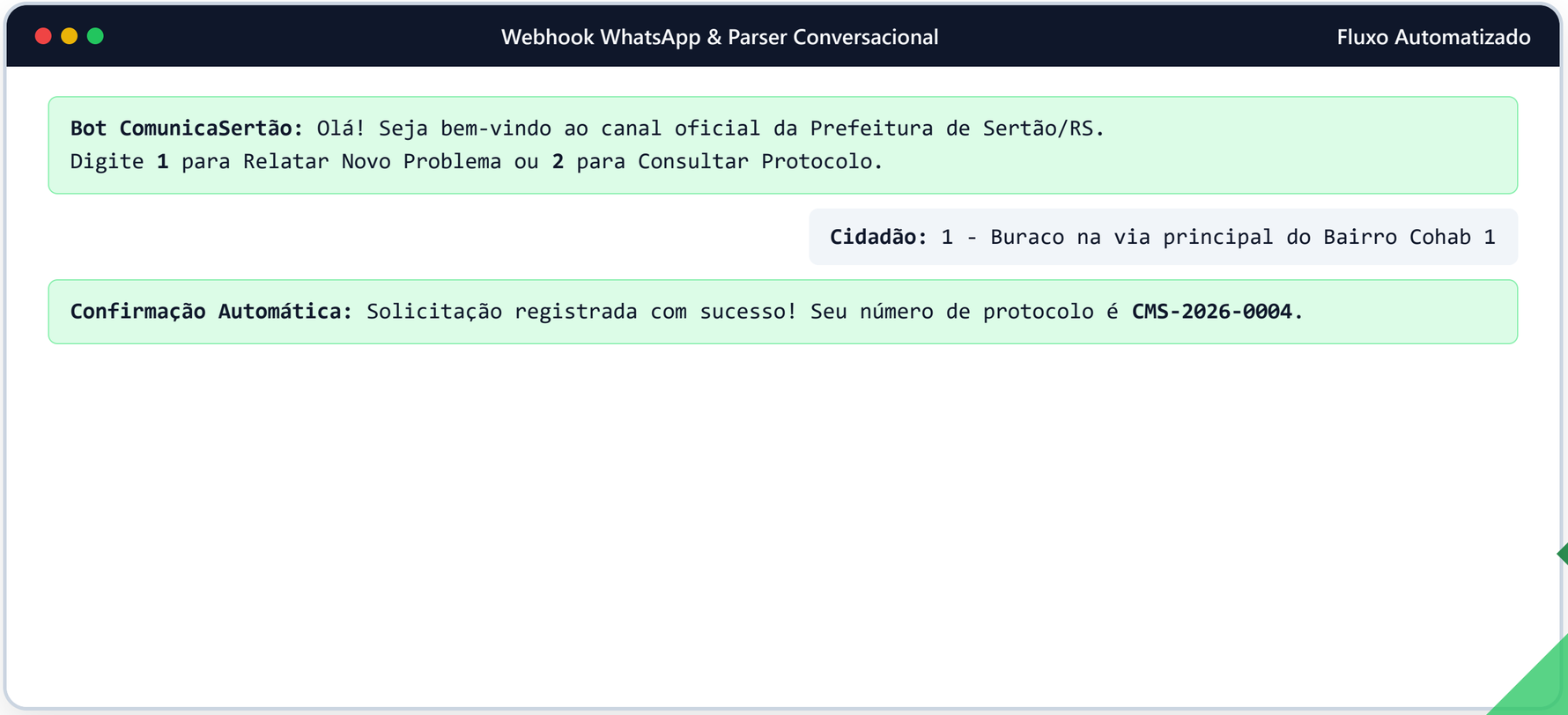
API REST - Gestão Operacional de Demandas — JSON Response


```
{
  "protocolo": "CMS-2026-0002",
  "cidadao_nome": "João Silva",
  "cidadao_telefone": "(54) 99876-5432",
  "categoria": "Pavimentação",
  "descricao": "Buraco na pista próximo à praça central",
  "localizacao": "Rua Flores da Cunha, Centro - Sertão/RS",
  "status": "Em Andamento",
  "prioridade": "ALTA",
  "criado_em": "2026-10-13T10:45:00"
}
```


Consultas e mutações estruturadas de chamados com serialização relacional via SQLAlchemy.

Prototipação e Implementação

Processamento de Webhook e Máquina de Estados



Recepção assíncrona de mensagens, condução de máquina de estados e geração de protocolo sequencial

Prototipação e Implementação

Upload de Evidências e Armazenamento Local

POST /upload/foto - Armazenamento de Arquivos — UploadFile & StaticFiles

```python
# Validação e Armazenamento Seguro de Evidências
ext = os.path.splitext(arquivo.filename)[1].lower()
if ext not in [".jpg", ".jpeg", ".png", ".webp"]:
    raise HTTPException(status_code=400, detail="Formato de imagem inválido")

nome_arquivo = f"foto_{uuid.uuid4().hex[:10]}{ext}"
caminho = os.path.join("uploads", nome_arquivo)
with open(caminho, "wb") as buffer:
    shutil.copyfileobj(arquivo.file, buffer)

return {"url": f"/uploads/{nome_arquivo}"}
```

Validação de formato, geração de identificador UUID e serviço estático local de fotos.

Prototipação e Implementação

Controle de Segurança e Aprovação de Servidores

Módulo de Governança de Usuários - SQLite — Administradores Homologados

Servidor	E-mail Institucional	Cargo	Status
Fabricio de Abreu	fabricio@sertao.rs.gov.br	Administrador	Aprovado
Patrick Telo Irschlinger	patrick@sertao.rs.gov.br	Administrador	Aprovado
Prefeitura Municipal	prefeitura@sertao.rs.gov.br	Administrador	Aprovado
Novo Servidor (Exemplo)	servidor@sertao.rs.gov.br	Servidor Municipal	Pendente

Controle de privilégios e validação hierárquica para novos servidores públicos.

5 Principais Requisitos e RNFs

1. RF01 / RF02 — Recepção Webhook e Máquina de Estados	Vinculado ao RNF01 (Baixa Latência e Desempenho)
2. RF03 — Emissão e Rastreabilidade de Protocolos	Vinculado ao RNF02 (Integridade Transacional ACID)
3. RF05 — Endpoints REST de Ocorrências e Filtros	Vinculado ao RNF01 (Tempo de Resposta Ágil)
4. RF06 — Autenticação e Aprovação de Servidores	Vinculado ao RNF03 (Segurança e Controle Institucional)
5. RF07 — Documentação Swagger UI e Validação Pydantic	Vinculado ao RNF03 (Validação e Padronização OpenAPI)

Trabalhos Futuros e Conclusão

Deploy em Nuvem

Publicação do back-end em servidor dedicado na nuvem com certificado digital SSL/HTTPS e domínio governamental oficial.

Meta Cloud API

Conexão ativa do webhook de produção com o chip telefônico oficial da Prefeitura Municipal de Sertão.

Notificações Ativas

Disparo automático de mensagens no WhatsApp do cidadão a cada avanço no status de seu protocolo cadastrado.